

Weekly Submission Document || Week-5

Trainee Information

Name : Golla Amulya

Email: gollaamulya418@gmail.com

Batch Name : B1_Python

Course : uGE_DotNet FSD (Python)

Trainer : Kunal Sir

16/03/2026

Practice Question 1: Employee Registration with Custom Validation

Requirements:

1. Create Employee model with:
 - Id
 - Name (Required, Min length 3)
 - Salary (Range 10,000–5,00,000)
 - Email (Required + Email format)
 - JoiningDate (Cannot be future date)
 2. If validation fails → show errors in view
 3. If valid → show Confirmation page
 4. Add custom validation logic inside controller to prevent future JoiningDate
-

● What You Must Practice:

- ✓ DataAnnotations
- ✓ Custom Validation Logic
- ✓ ModelState.AddModelError()
- ✓ Razor Validation Messages

Answer:**Employee.cs**

```
using System;
using System.ComponentModel.DataAnnotations;
namespace CollegePortalMvcApp.Models
{
    public class Employee
    {
        public int Id { get; set; }
        [Required(ErrorMessage = "Name is required")]
        [MinLength(3, ErrorMessage = "Name must be at least 3 characters")]
        public string Name { get; set; }
        [Range(10000, 500000, ErrorMessage = "Salary must be between 10,000 and 5,00,000")]
        public decimal Salary { get; set; }
        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Enter valid email")]
        public string Email { get; set; }
        [DataType(DataType.Date)]
        public DateTime JoiningDate { get; set; }
    }
}
```

EmployeeController.cs

```
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
using System;
namespace CollegePortalMvcApp.Controllers
{
```

```

public class EmployeeController : Controller
{
    public IActionResult Register()
    {
        return View();
    }
    [HttpPost]
    public IActionResult Register(Employee emp)
    {
        if (emp.JoiningDate > DateTime.Now)
        {
            ModelState.AddModelError("JoiningDate", "Joining date cannot be in the future");
        }
        if (ModelState.IsValid)
        {
            return View("Confirmation", emp);
        }
        return View(emp);
    }
}

```

Employee/Register.cshtml

```

@model CollegePortalMvcApp.Models.Employee
<h2>Employee Registration</h2>
<form asp-action="Register" method="post">
    <div>
        <label>ID</label>
        <input asp-for="Id" />
    </div>

```


<div>

<label>Name</label>

<input asp-for="Name" />

</div>

<div>

<label>Salary</label>

<input asp-for="Salary" />

</div>

<div>

<label>Email</label>

<input asp-for="Email" />

</div>

<div>

<label>Joining Date</label>

<input asp-for="JoiningDate" type="date"/>

</div>

<button type="submit">Register</button>

</form>

Employee/Confirmation.cshtml

@model CollegePortalMvcApp.Models.Employee

<h2>Employee Registered Successfully</h2>

<p>ID: @Model.Id</p>

<p>Name: @Model.Name</p>

<p>Salary: @Model.Salary</p>

<p>Email: @Model.Email</p>

<p>Joining Date: @Model.JoiningDate</p>

Practice Question 2: Product List with In-Memory Storage

Requirements:

1. Create Product model:
 - Id
 - Name
 - Price
 - Quantity
2. When user submits product:
 - Store it in a static List<Product>
 - Redirect to List page
3. Create:
 - Create View
 - Index View (show table of products)
4. Display products in HTML table using foreach

What You Must Practice:

- ✓ Static list (temporary storage)
- ✓ RedirectToAction()

- ✓ Passing List<Model> to View
- ✓ Razor foreach loop
- ✓ Table rendering

Answer:

Models/Product.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace CollegePortalMvcApp.Models
```

```
{  
    public class Product  
    {  
        public int Id { get; set; }  
        [Required]  
        public string Name { get; set; }  
        [Range(1, 100000)]  
        public decimal Price { get; set; }  
        [Range(1, 1000)]  
        public int Quantity { get; set; }  
    }  
}
```

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using CollegePortalMvcApp.Models;
```

```
using System.Collections.Generic;
```

```
namespace CollegePortalMvcApp.Controllers
```

```
{  
    public class ProductController : Controller  
    {  
        private static List<Product> products = new List<Product>();  
    }  
}
```

```

public IActionResult Index()
{
    return View(products);
}

public IActionResult Create()
{
    return View();
}

[HttpPost]
public IActionResult Create(Product product)
{
    if (ModelState.IsValid)
    {
        products.Add(product);
        return RedirectToAction("Index");
    }
    return View(product);
}
}
}

```

Views/Product/Create.cshtml

```

@model CollegePortalMvcApp.Models.Product
<h2>Add Product</h2>
<form asp-action="Create" method="post">
    <div>
        <label>Id</label>
        <input asp-for="Id"/>
    </div>

```

```
<br>
<div>
    <label>Name</label>
    <input asp-for="Name"/>
    <span asp-validation-for="Name"></span>
</div>
<br>
<div>
    <label>Price</label>
    <input asp-for="Price"/>
    <span asp-validation-for="Price"></span>
</div>
<br>
<div>
    <label>Quantity</label>
    <input asp-for="Quantity"/>
    <span asp-validation-for="Quantity"></span>
</div>
<br>
<button type="submit">Save Product</button>
</form>
<br>
<a href="/Product/Index">View Product List</a>
Views/Product/Index.cshtml
@model List<CollegePortalMvcApp.Models.Product>
<h2>Product List</h2>
<a href="/Product/Create">Add New Product</a>
```



```
<br><br>
<table border="1" cellpadding="10">
  <tr>
    <th>Id</th>
    <th>Name</th>
    <th>Price</th>
    <th>Quantity</th>
  </tr>
  @foreach (var p in Model)
  {
    <tr>
      <td>@p.Id</td>
      <td>@p.Name</td>
      <td>@p.Price</td>
      <td>@p.Quantity</td>
    </tr>
  }
</table>
```

Practice Question 3: Edit Feature (Without Database)

Requirements:

1. Use static List<Student>
2. Show all students in table
3. Add Edit button next to each
4. Clicking Edit should:
 - Load data into form
 - Update the record
 - Redirect to list

● **What You Must Practice:**

- ✓ Route parameter
- ✓ Passing id in URL
- ✓ HttpGet Edit(int id)
- ✓ HttpPost Edit(Student student)
- ✓ Model binding with hidden field
- ✓ Updating list item

Answer:

Models/Student.cs

```
using System.ComponentModel.DataAnnotations;
namespace CollegePortalMvcApp.Models
{
    public class Student
    {
        public int Id { get; set; }
        [Required]
        public string Name { get; set; }
        [Range(18, 60)]
        public int Age { get; set; }
        [EmailAddress]
        public string Email { get; set; }
    }
}
```

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
using System.Collections.Generic;
using System.Linq;
```

```
namespace CollegePortalMvcApp.Controllers
{
    public class StudentController : Controller
    {
        private static List<Student> students = new List<Student>();

        public IActionResult Index()
        {
            return View(students);
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Student student)
        {
            if (ModelState.IsValid)
            {
                students.Add(student);
                return RedirectToAction("Index");
            }

            return View(student);
        }

        public IActionResult Edit(int id)
        {
            var student = students.FirstOrDefault(s => s.Id == id);
            return View(student);
        }
    }
}
```

```

    }

    [HttpPost]
    public IActionResult Edit(Student student)
    {
        var existingStudent = students.FirstOrDefault(s => s.Id == student.Id);
        if (existingStudent != null)
        {
            existingStudent.Name = student.Name;
            existingStudent.Age = student.Age;
            existingStudent.Email = student.Email;
        }
        return RedirectToAction("Index");
    }
}
}

```

Views/Student/Create.cshtml

```

@model CollegePortalMvcApp.Models.Student
<h2>Add Student</h2>
<form asp-action="Create" method="post">
    <label>ID</label>
    <input asp-for="Id"/>
    <br><br>
    <label>Name</label>
    <input asp-for="Name"/>
    <span asp-validation-for="Name"></span>
    <br><br>
    <label>Age</label>
    <input asp-for="Age"/>

```

```

<span asp-validation-for="Age"></span>
<br><br>
<label>Email</label>
<input asp-for="Email"/>
<span asp-validation-for="Email"></span>
<br><br>
<button type="submit">Save Student</button>
</form>
<br>
<a href="/Student/Index">View Students</a>

```

Views/Student/Index.cshtml

```

@model List<CollegePortalMvcApp.Models.Student>
<h2>Student List</h2>
<a href="/Student/Create">Add Student</a>
<br><br>
<table border="1" cellpadding="10">
<tr>
<th>Id</th>
<th>Name</th>
<th>Age</th>
<th>Email</th>
<th>Edit</th>
</tr>
@foreach (var s in Model)
{
<tr>
<td>@s.Id</td>
<td>@s.Name</td>

```

```

<td>@s.Age</td>
<td>@s.Email</td>
<td>
<a href="/Student/Edit/@s.Id">Edit</a>
</td>
</tr>
}
</table>

```

Views/Student/Edit.cshtml

```

@model CollegePortalMvcApp.Models.Student
<h2>Edit Student</h2>
<form asp-action="Edit" method="post">
<input type="hidden" asp-for="Id"/>
<label>Name</label>
<input asp-for="Name"/>
<br><br>
<label>Age</label>
<input asp-for="Age"/>
<br><br>
<label>Email</label>
<input asp-for="Email"/>
<br><br>
<button type="submit">Update</button>
</form>

```

Practice Question 4: Conditional View Rendering

Requirements:

1. Create Order model:

- OrderId
- CustomerName
- Amount
- IsPremiumCustomer (checkbox)

2. In Details View:

- If IsPremiumCustomer = true → Show:

"Premium Discount Applied 

- If Amount > 5000 → Show:

"Eligible for Free Shipping"

Use Razor conditionals.

● What You Must Practice:

- ✓ Razor if condition
- ✓ Boolean checkbox binding
- ✓ Dynamic UI rendering
- ✓ Conditional HTML rendering

Example hint:

```
@if(Model.Amount > 5000)
{
    <p>Eligible for Free Shipping</p>
}
```

Answer:

Models/Order.cs

```
using System.ComponentModel.DataAnnotations;
namespace CollegePortalMvcApp.Models
{
    public class Order
    {
        public int OrderId { get; set; }
```

```

    [Required]
    public string? CustomerName { get; set; }
    public decimal Amount { get; set; }
    public bool IsPremiumCustomer { get; set; }
}
}

```

Controllers/OrderController.cs

```

using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
namespace CollegePortalMvcApp.Controllers
{
    public class OrderController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Create(Order order)
        {
            return View("Details", order);
        }
    }
}

```

Views/Order/Create.cshtml

```
@model CollegePortalMvcApp.Models.Order
```



```

<h2>Create Order</h2>
<form asp-action="Create" method="post">
    <label>Order ID</label>
    <input asp-for="OrderId"/>
    <br><br>
    <label>Customer Name</label>
    <input asp-for="CustomerName"/>
    <br><br>
    <label>Amount</label>
    <input asp-for="Amount"/>
    <br><br>
    <label>Premium Customer</label>
    <input asp-for="IsPremiumCustomer" type="checkbox"/>
    <br><br>
    <button type="submit">Submit Order</button>
</form>

```

Views/Order/Details.cshtml

```

@model CollegePortalMvcApp.Models.Order
<h2>Order Details</h2>
<p><b>Order ID:</b> @Model.OrderId</p>
<p><b>Customer:</b> @Model.CustomerName</p>
<p><b>Amount:</b> @Model.Amount</p>
<p><b>Premium Customer:</b> @Model.IsPremiumCustomer</p>
<hr>
@if (Model.IsPremiumCustomer)
{
    <p style="color:green">
        Premium Discount Applied 🎁
    </p>
}

```

```
</p>
}
@if(Model.Amount > 5000)
{
    <p style="color:blue">
        Eligible for Free Shipping
    </p>
}
```

17/3/2026 - Task

🔖 1. Dynamic Form Binding (List of Objects)

🧠 Problem:

Create a form where user can enter multiple employees dynamically.

Requirements:

- Model:

```
public class Employee
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

- User should be able to submit:

Employee 1 → Name, Age

Employee 2 → Name, Age

Employee 3 → Name, Age

📌 Task:

- Bind data to List<Employee>
- Display all employees in next view

👤 What You Practice:

- ✓ List model binding
- ✓ Index-based naming
- ✓ Razor loops

Answer:

Models/Employee.cs

```
namespace CollegePortalMvcApp.Models
{
    public class Employee
    {
        public string? Name { get; set; }
        public int Age { get; set; }
    }
}
```

Controllers/EmployeeController.cs

```
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
using System.Collections.Generic;
namespace CollegePortalMvcApp.Controllers
{
    public class EmployeeListController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
```

```

    public IActionResult Create(List<Employee> employees)
    {
        return View("Display", employees);
    }
}

```

Views/EmployeeList/Create.cshtml

```

@model List<CollegePortalMvcApp.Models.Employee>
<h2>Enter Employees</h2>
<form asp-action="Create" method="post">
    <h4>Employee 1</h4>
    <input name="employees[0].Name" placeholder="Name" />
    <input name="employees[0].Age" placeholder="Age" />
    <br><br>
    <h4>Employee 2</h4>
    <input name="employees[1].Name" placeholder="Name" />
    <input name="employees[1].Age" placeholder="Age" />
    <br><br>
    <h4>Employee 3</h4>
    <input name="employees[2].Name" placeholder="Name" />
    <input name="employees[2].Age" placeholder="Age" />
    <br><br>
    <button type="submit">Submit</button>
</form>

```

Views/EmployeeList/Display.cshtml

```

@model List<CollegePortalMvcApp.Models.Employee>
<h2>Employee List</h2>

```

```
<table border="1" cellpadding="10">
<tr>
  <th>Name</th>
  <th>Age</th>
</tr>
@foreach (var emp in Model)
{
  <tr>
    <td>@emp.Name</td>
    <td>@emp.Age</td>
  </tr>
}
</table>
```

🚩 2. Custom Validation Without DataAnnotations

🧠 Problem:

Create a form:

```
public class User
{
  public string Username { get; set; }
  public string Password { get; set; }
}
```

📋 Task:

- Username must NOT contain "admin"
 - Password must be at least 8 characters
 - Do NOT use DataAnnotations
-
-

 Bonus:

- **Show error messages in Razor view**

Answer:

Models/User.cs

```
namespace EmployeeListApp.Models
{
    public class User
    {
        public string? Username { get; set; }
        public string? Password { get; set; }
    }
}
```

Controllers/UserController.cs

```
using Microsoft.AspNetCore.Mvc;
using EmployeeListApp.Models;
namespace EmployeeListApp.Controllers
{
    public class UserController : Controller
    {
        public IActionResult Register()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Register(User user)
        {
            if (user.Username != null && user.Username.ToLower().Contains("admin"))
```

```

    {
        ModelState.AddModelError("Username", "Username cannot contain 'admin'");
    }
    if (user.Password == null || user.Password.Length < 8)
    {
        ModelState.AddModelError("Password", "Password must be at least 8
characters");
    }
    if (ModelState.IsValid)
    {
        return View("Success", user);
    }
    return View(user);
}
}
}

```

Views/User/Register.cshtml

```
@model EmployeeListApp.Models.User
```

```
<h2>User Registration</h2>
```

```
<form asp-action="Register" method="post">
```

```
    <div>
```

```
        <label>Username</label>
```

```
        <input asp-for="Username" />
```

```
        <span asp-validation-for="Username" style="color:red"></span>
```

```
    </div>
```

```
    <br>
```

```
    <div>
```

```
        <label>Password</label>
```

```
<input asp-for="Password" type="password"/>
<span asp-validation-for="Password" style="color:red"></span>
</div>
<br>
<button type="submit">Register</button>
</form>
```

Views/User/Success.cshtml

```
@model EmployeeListApp.Models.User
<h2>Registration    Successful 🎉
<p><b>Username:</b> @Model.Username</p>
```

🔗 3. One Form → Multiple Actions (Advanced)

🎯 Problem:

Create a form with:

- Save button
 - Preview button
-

📋 Task:

- If "Preview" clicked → show data in another view WITHOUT saving
 - If "Save" clicked → store in static list
-
-

💡 Hint:

```
<button name="action" value="preview">
<button name="action" value="save">
```

🔑 What You Practice:

✓ Form behavior

✓ Conditional logic in controller

Answer:

Models/Product.cs

```
namespace EmployeeListApp.Models
```

```
{  
    public class Product  
    {  
        public int Id { get; set; }  
        public string? Name { get; set; }  
        public decimal Price { get; set; }  
    }  
}
```

Controllers/ProductActionController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using EmployeeListApp.Models;
```

```
using System.Collections.Generic;
```

```
namespace EmployeeListApp.Controllers
```

```
{  
    public class ProductActionController : Controller  
    {  
        private static List<Product> products = new List<Product>();  
        public IActionResult Create()  
        {  
            return View();  
        }  
        [HttpPost]  
        public IActionResult Create(Product product, string action)
```

```

{
    if (action == "preview")
    {
        return View("Preview", product);
    }
    if (action == "save")
    {
        products.Add(product);
        return RedirectToAction("List");
    }
    return View(product);
}

public IActionResult List()
{
    return View(products);
}
}

```

Views/ProductAction/Create.cshtml

```

@model EmployeeListApp.Models.Product
<h2>Product Form</h2>
<form asp-action="Create" method="post">
    <label>Id</label>
    <input asp-for="Id"/>
    <br><br>
    <label>Name</label>
    <input asp-for="Name"/>
    <br><br>

```

```
<label>Price</label>
<input asp-for="Price"/>
<br><br>
<button type="submit" name="action" value="preview">Preview</button>
<button type="submit" name="action" value="save">Save</button>
</form>
```

Views/ProductAction/Preview.cshtml

```
@model EmployeeListApp.Models.Product
<h2>Preview Product</h2>
<p><b>Id:</b> @Model.Id</p>
<p><b>Name:</b> @Model.Name</p>
<p><b>Price:</b> @Model.Price</p>
<a href="/ProductAction/Create">Back</a>
```

Views/ProductAction/List.cshtml

```
@model List<EmployeeListApp.Models.Product>
<h2>Saved Products</h2>
<a href="/ProductAction/Create">Add New</a>
<br><br>
<table border="1">
<tr>
<th>Id</th>
<th>Name</th>
<th>Price</th>
</tr>
@foreach (var p in Model)
{
<tr>
<td>@p.Id</td>
```

```
<td>@p.Name</td>
<td>@p.Price</td>
</tr>
}
</table>
```

🔗 4. Edit Without Database (Real Challenge)

🔗 Problem:

Use static list:

```
static List<Product> products = new List<Product>();
```

🔗 Task:

- Show products in table
 - Add Edit button
 - Clicking Edit:
 - Loads data into form
 - Updates product
-
-

🗣️ What You Practice:

- ✓ GET + POST Edit
- ✓ Route parameter
- ✓ Updating object in list

Answer:

Models/Product.cs

```
namespace EmployeeListApp.Models
{
    public class Product
```

```

{
    public int Id { get; set; }
    public string? Name { get; set; }
    public decimal Price { get; set; }
}
}

```

Controllers/ProductController.cs

```

using Microsoft.AspNetCore.Mvc;
using EmployeeListApp.Models;
using System.Collections.Generic;
using System.Linq;
namespace EmployeeListApp.Controllers
{
    public class ProductController : Controller
    {
        private static List<Product> products = new List<Product>();
        public IActionResult Index()
        {
            return View(products);
        }
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Create(Product product)
        {
            products.Add(product);

```

```

        return RedirectToAction("Index");
    }

    public IActionResult Edit(int id)
    {
        var product = products.FirstOrDefault(p => p.Id == id);
        return View(product);
    }

    [HttpPost]
    public IActionResult Edit(Product product)
    {
        var existing = products.FirstOrDefault(p => p.Id == product.Id);
        if (existing != null)
        {
            existing.Name = product.Name;
            existing.Price = product.Price;
        }
        return RedirectToAction("Index");
    }
}

```

Views/Product/Create.cshtml

```

@model EmployeeListApp.Models.Product
<h2>Add Product</h2>
<form asp-action="Create" method="post">
    <label>Id</label>
    <input asp-for="Id"/>
    <br><br>
    <label>Name</label>

```

```
<input asp-for="Name"/>
<br><br>
<label>Price</label>
<input asp-for="Price"/>
<br><br>
<button type="submit">Save</button>
</form>
```

Views/Product/Index.cshtml

```
@model List<EmployeeListApp.Models.Product>
<h2>Product List</h2>
<a href="/Product/Create">Add Product</a>
<br><br>
<table border="1" cellpadding="10">
<tr>
<th>Id</th>
<th>Name</th>
<th>Price</th>
<th>Edit</th>
</tr>
@foreach (var p in Model)
{
<tr>
<td>@p.Id</td>
<td>@p.Name</td>
<td>@p.Price</td>
<td>
<a href="/Product/Edit/@p.Id">Edit</a>
</td>
</tr>
}
```

```
</tr>
}
</table>
```

Views/Product/Edit.cshtml

```
@model EmployeeListApp.Models.Product
<h2>Edit Product</h2>
<form asp-action="Edit" method="post">
  <!-- Hidden ID (VERY IMPORTANT) -->
  <input type="hidden" asp-for="Id" />
  <label>Name</label>
  <input asp-for="Name"/>
  <br><br>
  <label>Price</label>
  <input asp-for="Price"/>
  <br><br>
  <button type="submit">Update</button>
</form>
```

5. Preserve Form Data After Validation Error

Problem:

- Form has:
 - Name
 - Gender (dropdown)
 - Skills (checkbox list)
-

Task:

- If validation fails:
 - Previously selected values **MUST** remain selected

👤 What You Practice:

✓ ModelState

✓ UI state persistence

✓ Real-world UX handling

Answer:

Models/UserForm.cs

```
using System.Collections.Generic;
namespace EmployeeListApp.Models
{
    public class UserForm
    {
        public string? Name { get; set; }
        public string? Gender { get; set; }
        public List<string>? Skills { get; set; }
    }
}
```

Controllers/UserFormController.cs

```
using Microsoft.AspNetCore.Mvc;
using EmployeeListApp.Models;
using System;
namespace EmployeeListApp.Controllers
{
    public class UserFormController : Controller
    {
        public IActionResult Create()
        {

```

```

        return View();
    }

    [HttpPost]
    public IActionResult Create(UserForm user)
    {
        if (string.IsNullOrEmpty(user.Name))
        {
            ModelState.AddModelError("Name", "Name is required");
        }

        if (string.IsNullOrEmpty(user.Gender))
        {
            ModelState.AddModelError("Gender", "Select gender");
        }

        if (user.Skills == null || user.Skills.Count == 0)
        {
            ModelState.AddModelError("Skills", "Select at least one skill");
        }

        if (ModelState.IsValid)
        {
            return View("Success", user);
        }

        return View(user);
    }
}

```

Views/UserForm/Create.cshtml

@model EmployeeListApp.Models.UserForm

<h2>User Form</h2>

<form asp-action="Create" method="post">

<div>

<label>Name</label>

<input asp-for="Name" />

</div>

<div>

<label>Gender</label>

<select asp-for="Gender">

<option value="">--Select--</option>

<option value="Male">Male</option>

<option value="Female">Female</option>

</select>

</div>

<div>

<label>Skills</label>

<input type="checkbox" name="Skills" value="Java"

@(Model?.Skills != null CC Model.Skills.Contains("Java") ? "checked" : "") /> Java

<input type="checkbox" name="Skills" value="C#"

@(Model?.Skills != null CC Model.Skills.Contains("C#") ? "checked" : "") /> C#

<input type="checkbox" name="Skills" value="Python"

@(Model?.Skills != null CC Model.Skills.Contains("Python") ? "checked" : "") />


```
</div>

<br>

<button type="submit">Submit</button>

</form>
```

Views/UserForm/Success.cshtml

@model EmployeeListApp.Models.UserForm

```
<h2>Form Submitted Successfully 🎉

<p><b>Name:</b> @Model.Name</p>

<p><b>Gender:</b> @Model.Gender</p>

<p><b>Skills:</b> @string.Join(" ", Model.Skills)</p>
```

Scenario

1. Model Binding with Complex Object (Hidden Trap)

Scenario:

public class Address

```
{
    public string City { get; set; }
    public string State { get; set; }
}
```

public class Employee

```
{
    public string Name { get; set; }
    public Address Address { get; set; }
}
```

View:

```
<form method="post">
    <input type="text" name="Name" />
    <input type="text" name="City" />
    <input type="text" name="State" />
    <button type="submit">Submit</button>
</form>
```

Controller:

[HttpPost]

```
public IActionResult Create(Employee emp)  
{  
    return Content(emp.Address.City);  
}
```

🔗 Question:

Why is emp.Address null?

Answer:

emp.Address is null because model binding requires matching name structure for nested objects.

🔗 2. Multiple Submit Buttons Confusion

🔗 Scenario:

```
<form method="post">  
    <button type="submit" name="action" value="save">Save</button>  
    <button type="submit" name="action" value="delete">Delete</button>  
</form>
```

Controller:

[HttpPost]

```
public IActionResult Process(string action)  
{  
    if (action == "save")  
        return Content("Saved");  
  
    if (action == "delete")  
        return Content("Deleted");  
  
    return Content("Unknown");  
}
```

🔗 Question:

How does MVC know which button was clicked?

Answer:

MVC knows which button was clicked because both buttons share the same name but different values.

🔗 3. Why ModelState.IsValid is FALSE (Even When Input is Correct)

🔗 Scenario:

```
public class User  
{  
  public int Age { get; set; }  
}
```

View:

```
<form method="post">  
  <input type="text" name="Age" value="twenty" />  
  <button type="submit">Submit</button>  
</form>
```

🔗 Question:

Why is ModelState.IsValid == false even without validation attributes?

Answer:

It becomes false because model binding fails to convert "twenty" into an integer.

🔗 4. View Not Updating After POST (Classic Trap)

🔗 Scenario:

```
[HttpPost]  
public IActionResult Create(Product p)  
{  
  p.Name = "Modified";
```

```
return View();  
}
```

View:

```
<p>@Model.Name</p>
```

 Question:

Why does it show null instead of "Modified"?

. Missing Data in POST (Checkbox Trap)

Answer:

It shows null because you are returning View() without passing the model.

 5 Scenario:

public class User

```
{  
    public string Name { get; set; }  
    public bool IsSubscribed { get; set; }  
}
```

View:

```
<form method="post">  
    <input type="text" name="Name" />  
    <input type="checkbox" name="IsSubscribed" />  
    <button type="submit">Submit</button>  
</form>
```

Controller:

[HttpPost]

```
public IActionResult Create(User user)  
{  
    return Content(user.IsSubscribed.ToString());  
}
```

 Question:

Why does IsSubscribed always come as false when checkbox is unchecked, and what happens internally during model binding?

Answer:

Checkbox sends value only when checked; if unchecked, nothing is sent. So the model property remains default (false/null). MVC cannot bind a value that is not present in the request

6. Default Value Overwritten (Hidden Bug)

Scenario:

```
public class Product  
{  
    public string Name { get; set; }  
    public int Quantity { get; set; } = 10;  
}
```

View:

```
<form method="post">  
    <input type="text" name="Name" />  
    <button type="submit">Submit</button>  
</form>
```

Controller:

```
[HttpPost]  
public IActionResult Create(Product p)  
{  
    return Content(p.Quantity.ToString());  
}
```

Question:

Why does Quantity become 0 instead of 10 after form submission?

Answer:

Quantity becomes 0 because model binding overrides default values.

7. Route Parameter vs Form Data Conflict

Scenario:

[HttpPost]

```
public IActionResult Edit(int id, Product p)  
{  
    return Content($"Route Id: {id}, Model Id: {p.Id}");  
}
```

View:

```
<form method="post" action="/Product/Edit/5">  
    <input type="hidden" name="Id" value="10" />  
    <button type="submit">Submit</button>  
</form>
```

Question:

What values will be printed and why is there a mismatch?

Answer:

Output: Route Id: 5, Model Id: 10
id comes from route while p.Id comes from form (hidden input = 10).
MVC binds them separately, causing mismatch.

8. ViewBag Lost After Redirect (Common Trap)

Scenario:

```
public IActionResult Create()  
{  
    ViewBag.Message = "Hello";  
    return RedirectToAction("Index");  
}
```

```
public IActionResult Index()  
{  
    return View();  
}
```

View:

```
<p>@ViewBag.Message</p>
```

Question:

Why is ViewBag.Message null in the Index view?

Answer:

ViewBag is null because RedirectToAction creates a new request.

18/03/2026

Practice Question 1 — Library Management System

Scenario

Build a **Library Management Module** inside the same project structure.

Requirements

Models

1 Book

- Id
- Title (Required)
- Author (Required)
- Price (Range 100–5000)
- CategoryId (FK)
- Category (Navigation Property)

3 Category

- Id
 - Name (Required)
 - ICollection<Book>
-
-

Features To Implement

■ 1. Full CRUD for Books

- Create
- List
- Edit
- Delete

■ 2. Show Category Name in Book List

Use:

`.Include(b => b.Category)`

■ 3. LINQ Queries

On Book Index page:

- Show total books
 - Show most expensive book
 - Order books by price descending
-

■ 4. Routing Practice

Create custom route:

```
app.MapControllerRoute(  
    name: "booksByCategory",  
    pattern: "Books/Category/{id}",  
    defaults: new { controller = "Book", action = "ByCategory" });
```

Create action:

```
public IActionResult ByCategory(int id)
```

■ 5. State Management

- Use TempData for success message
 - Store last added book in Session
-

● What This Tests

- ✓ Relationship (One-to-Many)
- ✓ Foreign Key
- ✓ Eager Loading
- ✓ Custom Routing
- ✓ LINQ Aggregation
- ✓ Session
- ✓ TempData

Answer:

Models/Category.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentMvcApp.Models
{
    public class Category
    {
        public int Id { get; set; }

        [Required]

        public string Name { get; set; }

        public ICollection<Book> Books { get; set; } = new List<Book>();
    }
}
```

Models/Book.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentMvcApp.Models
{
    public class Book
    {
        public int Id { get; set; }

        [Required]
```

```

    public string Title { get; set; }

    [Required]

    public string Author { get; set; }

    [Range(100, 5000)]

    public int Price { get; set; }

    public int CategoryId { get; set; }

    public Category? Category { get; set; }
}
}

```

Controllers/BookController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentMvcApp.Data;
using StudentMvcApp.Models;
namespace StudentMvcApp.Controllers
{
    public class BookController : Controller
    {
        private readonly ApplicationDbContext _context;

        public BookController(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<IActionResult> Index()
        {
            var books = await _context.Books
                .Include(b => b.Category)
                .OrderByDescending(b => b.Price)

```

```

        .ToListAsync();
ViewBag.TotalBooks = books.Count;
if (books.Any())
    ViewBag.MaxPriceBook = books.MaxBy(b => b.Price)?.Title;
return View(books);
}

public IActionResult Create()
{
    ViewBag.Categories = _context.Categories.ToList();
    return View();
}

[HttpPost]
public async Task<IActionResult> Create(Book book)
{
    if (ModelState.IsValid)
    {
        _context.Add(book);
        await _context.SaveChangesAsync();
        TempData["msg"] = "Book Added!";
        HttpContext.Session.SetString("LastBook", book.Title);
        return RedirectToAction("Index");
    }

    ViewBag.Categories = _context.Categories.ToList();
    return View(book);
}

public async Task<IActionResult> Edit(int id)
{
    var book = await _context.Books.FindAsync(id);

```

```

        ViewBag.Categories = _context.Categories.ToList();
        return View(book);
    }

    [HttpPost]
    public async Task<IActionResult> Edit(Book book)
    {
        _context.Update(book);
        await _context.SaveChangesAsync();
        TempData["msg"] = "Updated!";
        return RedirectToAction("Index");
    }

    public async Task<IActionResult> Delete(int id)
    {
        var book = await _context.Books.FindAsync(id);
        if (book != null)
        {
            _context.Remove(book);
            await _context.SaveChangesAsync();
        }
        return RedirectToAction("Index");
    }

    public IActionResult ByCategory(int id)
    {
        var books = _context.Books
            .Include(b => b.Category)
            .Where(b => b.CategoryId == id)
            .ToList();
    }

```

```

        return View("Index", books);
    }
}
}

```

Views/Book/Index.cshtml

```

@model IEnumerable<StudentMvcApp.Models.Book>

<h2>Book List</h2>

@if (TempData["msg"] != null)
{
    <p style="color:green">@TempData["msg"]</p>
}

<p>Total Books: @ViewBag.TotalBooks</p>
<p>Most Expensive: @ViewBag.MaxPriceBook</p>
@if (Context.Session.GetString("LastBook") != null)
{
    <p>Last Added: @Context.Session.GetString("LastBook")</p>
}

<a asp-action="Create">Add Book</a>

<table border="1">
<tr>
<th>Title</th>
<th>Author</th>
<th>Price</th>
<th>Category</th>
<th>Action</th>
</tr>

@foreach (var b in Model)
{

```



```
<tr>
<td>@b.Title</td>
<td>@b.Author</td>
<td>@b.Price</td>
<td>@b.Category?.Name</td>
<td>
<a asp-action="Edit" asp-route-id="@b.Id">Edit</a> |
<a asp-action="Delete" asp-route-id="@b.Id">Delete</a>
</td>
</tr>
}
</table>
```

Views/Book/Create.cshtml

```
@model StudentMvcApp.Models.Book
<h2>Create Book</h2>
<form asp-action="Create" method="post">
<div>
<label>Title</label>
<input asp-for="Title" />
</div>
<div>
<label>Author</label>
<input asp-for="Author" />
</div>
<div>
<label>Price</label>
<input asp-for="Price" />
</div>
```

```
<div>
<label>Category</label>
<select asp-for="CategoryId">
@foreach (var c in ViewBag.Categories)
{
<option value="@c.Id">@c.Name</option>
}
</select>
</div>
<button type="submit">Save</button>
</form>
```

Views/Book/Edit.cshtml

```
@model StudentMvcApp.Models.Book
<h2>Edit Book</h2>
<form asp-action="Edit" method="post">
<input type="hidden" asp-for="Id" />
<input asp-for="Title" />
<input asp-for="Author" />
<input asp-for="Price" />
<select asp-for="CategoryId">
@foreach (var c in ViewBag.Categories)
{
<option value="@c.Id">@c.Name</option>
}
</select>
<button type="submit">Update</button>
</form>
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using StudentMvcApp.Data;
using StudentMvcApp.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext(options =>
options.UseSqlite("Data Source=library.db"));
builder.Services.AddSession();

var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseSession();
app.UseAuthorization();
app.MapControllerRoute(
name: "default",
pattern: "{controller=Book}/{action=Index}/{id?}");
app.MapControllerRoute(
name: "booksByCategory",
pattern: "Books/Category/{id}",
defaults: new { controller = "Book", action = "ByCategory" });
using (var scope = app.Services.CreateScope())
{
var context = scope.ServiceProvider.GetRequiredService();

context.Database.Migrate();

if (!context.Categories.Any())
{
context.Categories.AddRange(
    new Category { Name = "Programming" },
    new Category { Name = "Science" },
    new Category { Name = "Commerce" }
);

context.SaveChanges();
}
```

```
}  
app.Run();
```

Practice Question 2 — Online Store (Orders + Products)

Scenario

Create a small **E-Commerce Order Module**.

Models

1 Product

- Id
- Name (Required)
- Price
- StockQuantity

Order

- Id
 - OrderDate
 - TotalAmount
 - ProductId
 - Product (Navigation Property)
-

Features To Implement

1. Create Order Page

- Dropdown of products
 - Auto calculate total amount
 - Decrease stock when order placed
-

2. LINQ Advanced

On Orders page:

- Show total revenue
- Show products with low stock (<5)
- Group orders by product

Example:

```
_context.Orders
    .GroupBy(o => o.Product.Name)
    .Select(g => new {
        Product = g.Key,
        Count = g.Count()
    });
```

3. Edit Order

- Update product
 - Recalculate stock properly
-

4. Performance Practice

Use:

```
.AsNoTracking()
```

For read-only queries.

5. Routing Practice

Create route:

```
/Orders/Details/5
```

Use:

```
public IActionResult Details(int id)
```

What This Tests

- ✓ Many-to-One Relationship
- ✓ Business Logic in Controller

- ✓ LINQ GroupBy
- ✓ Stock Management
- ✓ AsNoTracking
- ✓ Eager Loading
- ✓ Routing Parameters
- ✓ Real-world logic

Answer:

Models/Product.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentMvcApp.Models
{
    public class Product
    {
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }

        public int Price { get; set; }

        public int StockQuantity { get; set; }

        public ICollection<Order>? Orders { get; set; }
    }
}
```

Models/Order.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentMvcApp.Models
{
    public class Order
    {
        public int Id { get; set; }

        public DateTime OrderDate { get; set; } = DateTime.Now;

        public int TotalAmount { get; set; }

        public int ProductId { get; set; }

        public Product? Product { get; set; }
    }
}
```

```
}  
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;  
using StudentMvcApp.Models;  
  
namespace StudentMvcApp.Data  
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions options)  
            : base(options)  
        {  
  
            public DbSet<Product> Products { get; set; }  
            public DbSet<Order> Orders { get; set; }  
            public DbSet<Book> Books { get; set; }  
            public DbSet<Category> Categories { get; set; }  
  
        }  
    }  
}
```

Controllers/OrderController.cs

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using StudentMvcApp.Data;  
using StudentMvcApp.Models;  
  
namespace StudentMvcApp.Controllers  
{  
    public class OrderController : Controller  
    {  
        private readonly AppDbContext _context;  
  
        public OrderController(AppDbContext context)  
        {  
            _context = context;  
        }  
    }  
}
```

```

public IActionResult Index()
{
    var orders = _context.Orders
        .Include(o => o.Product)
        .AsNoTracking()
        .ToList();
    ViewBag.TotalRevenue = orders.Sum(o => o.TotalAmount);
    ViewBag.LowStock = _context.Products
        .Where(p => p.StockQuantity < 5)
        .ToList();
    ViewBag.Grouped = _context.Orders
        .Include(o => o.Product)
        .AsEnumerable()
        .GroupBy(o => o.Product.Name)
        .Select(g => new {
            Product = g.Key,
            Count = g.Count()
        }).ToList();
    return View(orders);
}

public IActionResult Create()
{
    ViewBag.Products = _context.Products.ToList();
    return View();
}

[HttpPost]
public IActionResult Create(Order order)

```



```

{
    var product = _context.Products.Find(order.ProductId);
    if (product != null && product.StockQuantity > 0)
    {
        order.TotalAmount = product.Price;
        product.StockQuantity--;
        _context.Orders.Add(order);
        _context.SaveChanges();
        TempData["msg"] = "Order placed!";
        return RedirectToAction("Index");
    }
    TempData["msg"] = "Out of stock!";
    return RedirectToAction("Create");
}

public IActionResult Edit(int id)
{
    var order = _context.Orders.Find(id);
    ViewBag.Products = _context.Products.ToList();
    return View(order);
}

[HttpPost]
public IActionResult Edit(Order order)
{
    var oldOrder = _context.Orders.AsNoTracking().First(o => o.Id == order.Id);
    var oldProduct = _context.Products.Find(oldOrder.ProductId);
    var newProduct = _context.Products.Find(order.ProductId);
    oldProduct.StockQuantity++;
    newProduct.StockQuantity--;
}

```

```

        order.TotalAmount = newProduct.Price;
        _context.Update(order);
        _context.SaveChanges();
        TempData["msg"] = "Order updated!";
        return RedirectToAction("Index");
    }

    public IActionResult Details(int id)
    {
        var order = _context.Orders
            .Include(o => o.Product)
            .FirstOrDefault(o => o.Id == id);
        return View(order);
    }
}
}
}

```

Views/Order/Index.cshtml

```

@model IEnumerable<StudentMvcApp.Models.Order>
<h2>Orders</h2>
<p>Total Revenue: @ViewBag.TotalRevenue</p>
<h4>Low Stock Products</h4>
<ul>
    @foreach (var p in ViewBag.LowStock)
    {
        <li>@p.Name (@p.StockQuantity)</li>
    }
</ul>
<h4>Orders Grouped</h4>
<ul>

```

```

@foreach(var g in ViewBag.Grouped)
{
    <li>@g.Product - @g.Count orders</li>
}
</ul>
<a asp-action="Create">Create Order</a>
<table border="1">
<tr><th>Product</th><th>Amount</th><th>Action</th></tr>
@foreach(var o in Model)
{
    <tr>
    <td>@o.Product?.Name</td>
    <td>@o.TotalAmount</td>
    <td>
    <a asp-action="Edit" asp-route-id="@o.Id">Edit</a> |
    <a asp-action="Details" asp-route-id="@o.Id">Details</a>
    </td>
    </tr>
}
</table>

```

Views/Order/Create.cshtml

```

@model StudentMvcApp.Models.Order
<h2>Create Order</h2>
<form method="post">
<select asp-for="ProductId">
@foreach(var p in ViewBag.Products)
{

```

```
<option value="@p.Id">@p.Name - ₹@p.Price</option>
}
</select>
<button type="submit">Place Order</button>
</form>
```

Views/Order/Edit.cshtml

```
@model StudentMvcApp.Models.Order
<form method="post">
<input type="hidden" asp-for="Id" />
<select asp-for="ProductId">
@foreach(var p in ViewBag.Products)
{
<option value="@p.Id">@p.Name</option>
}
</select>
<button type="submit">Update</button>
</form>
```

Views/Order/Details.cshtml

```
@model StudentMvcApp.Models.Order
<h2>Order Details</h2>
<p>Product: @Model.Product?.Name</p>
<p>Amount: @Model.TotalAmount</p>
<p>Date: @Model.OrderDate</p>
```

◆ 1. Hello MVC Flow

Task:

Create a HomeController with action Index() that returns a View showing "Welcome to ASP.NET Core 8".

Concept:

- How Controller → View flow works (MVC basics)

Answer:

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Views/Home/Index.cshtml

```
<h2>Welcome to ASP.NET Core 8</h2>
```

◆ 2. Passing Data (ViewBag)**Task:**

Pass a string message from controller to view using ViewBag.

Concept:

- Dynamic data passing (no model)

Answer:

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```

namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            ViewBag.Message = "Hello from Controller!";
            return View();
        }
    }
}

```

Index.cshtml

```

@{
    ViewData["Title"] = "Home Page";
}

<h2>Welcome to ASP.NET Core 8</h2>

<p>@ViewBag.Message</p>

```

◆ 3. Strongly Typed Model

Task:

Create a Student model (Id, Name, Age) and display it in a view.

Concept:

- Strongly typed views (@model)

Answer:

Student.cs

```

namespace StudentMvcApp.Models
{
    public class Student

```

```

{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
}

```

HomeController.cs

```

using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Models;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult StudentDetails()
        {
            var student = new Student
            {
                Id = 1,
                Name = "Sonali",
                Age = 21
            };
            return View(student);
        }
    }
}

```

StudentDetails.cshtml

```

@model StudentMvcApp.Models.Student
<h2>Student Details</h2>
<p>ID: @Model.Id</p>

```

<p>Name: @Model.Name</p>

<p>Age: @Model.Age</p>

◆ 4. Multiple Data to View

Task:

Send a list of students from controller and display using @foreach.

Concept:

— Collections + Razor looping

Answer:

```
public IActionResult StudentList()
{
    var students = new List<Student>
    {
        new Student { Id = 1, Name = "Sonali", Age = 21 },
        new Student { Id = 2, Name = "Rahul", Age = 17 },
        new Student { Id = 3, Name = "Amit", Age = 25 }
    };
    return View(students);
}
```

StudentList.cshtml

```
@model List<StudentMvcApp.Models.Student>
<h2>Student List</h2>
<table border="1">
    <tr>
        <th>Name</th>
        <th>Age</th>
    </tr>
    @foreach (var s in Model)
    {
```



```
<tr>
  <td>@s.Name</td>
  <td>@s.Age</td>
</tr>
}
</table>
```

◆ 5. Razor Conditions

Task:

Display "Adult" if Age > 18 else "Minor".

Concept:

— @if in Razor

Answer:

```
@foreach (var s in Model)
{
  <tr>
    <td>@s.Name</td>
    <td>@s.Age</td>
    <td>
      @if (s.Age > 18)
      {
        <span>Adult</span>
      }
      else
      {
        <span>Minor</span>
      }
    </td>
  </tr>
}
```

```
</tr>
}
```

◆ 6. Form Handling (GET s POST)

Task:

Create a form to enter Name and Age. Submit it and display result.

Concept:

— GET vs POST requests

Answer:

```
public IActionResult Create()
{
    return View();
}

[HttpPost]
public IActionResult Create(Student s)
{
    return View("Result", s);
}
```

Create.cshtml

```
@model StudentMvcApp.Models.Student
<h2>Enter Student</h2>
<form method="post">
    <label>Name:</label>
    <input type="text" name="Name" />
    <br />
    <label>Age:</label>
    <input type="number" name="Age" />
    <br />
```

```
<button type="submit">Submit</button>
</form>
```

Result.cshtml

```
@model StudentMvcApp.Models.Student
<h2>Submitted Data</h2>
<p>Name: @Model.Name</p>
<p>Age: @Model.Age</p>
@if (Model.Age > 18)
{
    <p>Status: Adult</p>
}
else
{
    <p>Status: Minor</p>
}
```

◆ 7. Model Binding

Task:

Bind form data directly to a User model in POST method.

Concept:

— Automatic binding (magic of ASP.NET Core)

Answer:

Model Binding means ASP.NET automatically takes form data and fills it into a model object.

◆ 8. Model Validation

Task:

Add validation:

- Name → Required

- Age → 18–60

Show validation errors in UI.

Concept:

- Data Annotations + ModelState

Answer:

Model Validation means checking user input before processing it. We use Data Annotations like [Required], [Range]. If input is wrong then errors are stored in ModelState and shown in UI

◆ **G. ModelState Check**

Task:

If ModelState.IsValid == false, return same view with errors.

Concept:

- Server-side validation flow

Answer:

User.cs

```
using System.ComponentModel.DataAnnotations;
namespace StudentMvcApp.Models
{
    public class User
    {
        public int Id { get; set; }
        [Required(ErrorMessage = "Name is required")]
        public string Name { get; set; }
        [Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]
        public int Age { get; set; }
    }
}
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Models;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
```

```

{
    public IActionResult Register()
    {
        return View();
    }
    [HttpPost]
    public IActionResult Register(User user)
    {
        if (!ModelState.IsValid)
        {
            return View(user);        }
        return View("Success", user);
    }
}
}

```

Register.cshtml

```

@model StudentMvcApp.Models.User
<h2>User Registration</h2>
<form method="post">
    <div>
        <label>Name</label>
        <input asp-for="Name" />
        <span asp-validation-for="Name" style="color:red"></span>
    </div>
    <br />
    <div>
        <label>Age</label>
        <input asp-for="Age" />
        <span asp-validation-for="Age" style="color:red"></span>
    </div>
    <br />
    <button type="submit">Submit</button>
</form>
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Success.cshtml

```

@model StudentMvcApp.Models.User
<h2>Registration Successful h2>

```

<p>Name: @Model.Name</p>

<p>Age: @Model.Age</p>

◆ 10. Custom Routing

Task:

Create route:

/product/details/5 → display product id

Concept:

- URL mapping (routing system)

Answer:

Custom Routing is used to define our own URL pattern instead of default routing. It maps a specific URL → Controller → Action method.

```
app.MapControllerRoute(
    name: "productDetails",
    pattern: "product/details/{id}",
    defaults: new { controller = "Product", action = "Details" }
);
```

ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class ProductController : Controller
    {
        public IActionResult Details(int id)
        {
            return Content("Product ID: " + id);
        }
    }
}
```

◆ 11. Attribute Routing

Task:

Use [Route("home/about")] in controller.

Concept:

— Clean URL design

Answer:**HomeController.cs**

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        [Route("home/about")]
        public IActionResult About()
        {
            return Content("This is About Page");
        }
    }
}
```

◆ 12. Session Usage**Task:**

Store username in session and display on another page.

Concept:

— Temporary user data storage

Answer:

Session is used to store temporary user data on server.

Program.cs

```
using Microsoft.EntityFrameworkCore;
using StudentMvcApp.Data;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddSession();
```

```
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseSession();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult SetUser()
        {
            HttpContext.Session.SetString("username", "Sonali");
            return Content("Username stored in session");
        }
        public IActionResult GetUser()
        {
            var name = HttpContext.Session.GetString("username");
            if (name == null)
            {
                return Content("No user found in session");
            }
            return Content("Username: " + name);
        }
    }
}
```


}

◆ 13. TempData Usage

Task:

After form submit, show success message using TempData.

Concept:

— One-time data transfer between requests

Answer:

TempData is used to pass data from one request to another. Mostly used after form submit → redirect → show message

Controller

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Create(string name)
        {
            TempData["Success"] = "Form submitted successfully!";
            return RedirectToAction("Success");
        }
        public IActionResult Success()
        {
            return View();
        }
    }
}
```

Create.cshtml

```
<h2>Enter Name</h2>
<form method="post">
```

```
<input type="text" name="name" />
<button type="submit">Submit</button>
</form>
```

Success.cshtml

```
<h2>Result</h2>
@if (TempData["Success"] != null)
{
    <p style="color:green">@TempData["Success"]</p>
}
```

◆ 14. Cookies

Task:

Store user theme preference (dark/light) in cookies.

Concept:

— Client-side storage

Answer:

Cookies store data on browser (client-side). Used for preferences like theme (dark/light)

Controller

```
using Microsoft.AspNetCore.Mvc;
namespace StudentMvcApp.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult SetTheme(string theme)
        {
            Response.Cookies.Append("theme", theme);
            return Content("Theme saved: " + theme);
        }
        public IActionResult GetTheme()
        {
            var theme = Request.Cookies["theme"];
            return Content("Theme: " + theme);
        }
    }
}
```

◆ 15. Dependency Injection (DI)

Task:

Create a service GreetingService and inject into controller.

Concept:

- Loose coupling + service usage

Answer:

DI means injecting services into controller instead of creating manually.

GreetingService.cs

```
namespace StudentMvcApp.Services
{
    public class GreetingService
    {
        public string GetMessage()
        {
            return "Hello from Greeting Service!";
        }
    }
}
```

Program.cs

```
using StudentMvcApp.Services;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<GreetingService>();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Services;
namespace StudentMvcApp.Controllers
```

```

{
    public class HomeController : Controller
    {
        private readonly GreetingService _greetingService;
        public HomeController(GreetingService greetingService)
        {
            _greetingService = greetingService;
        }
        public IActionResult Greeting()
        {
            var message = _greetingService.GetMessage();
            return Content(message);
        }
    }
}

```

◆ 16. Setup EF Core (DbContext)

Task:

Create ApplicationDbContext and connect to SQL Server.

Concept:

— Database connection + ORM setup

Answer:

EF Core is used to connect C# code with database. DbContext acts as a bridge between application and SQL Server.

Student.cs

```

using System.ComponentModel.DataAnnotations;
namespace StudentMvcApp.Models
{
    public class Student
    {
        public int Id { get; set; }
        [Required]
        public string Name { get; set; }
        public int Age { get; set; }
    }
}

```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using StudentMvcApp.Models;
namespace StudentMvcApp.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }
        public DbSet<Student> Students { get; set; }
    }
}
```

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=StudentDb;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Server== local SQL Server

Program.cs

```
using Microsoft.EntityFrameworkCore;
using StudentMvcApp.Data;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

◆ 17. Create Table (Migration)

Task:

Create Student table using migration.

Concept:

— Code First approach

Answer:

Migration means convert C# model → database table. Code First approach = write code → generate DB automatically.

Example Model

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

DbContext

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

Migration Commands

dotnet ef migrations add InitialCreate

dotnet ef database update

◆ 18. Insert Data (Create)

Task:

Add a new student record using EF Core.

Concept:

— Add() + SaveChanges()

Answer:

In EF Core, inserting data means adding a new record to the database table.

CONTROLLER

```
using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Data;
using StudentMvcApp.Models;
namespace StudentMvcApp.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;
        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Create(Student s)
        {
            _context.Students.Add(s);
            _context.SaveChanges();
            return RedirectToAction("Index");
        }
    }
}
```

VIEW (FORM)

```
@model StudentMvcApp.Models.Student
<h2>Add Student</h2>
<form method="post">
    <label>Name:</label>
    <input type="text" name="Name" />
    <br />
    <label>Age:</label>
```

```
<input type="number" name="Age" />
<br />
<button type="submit">Save</button>
</form>
```

◆ 1G. Read Data (LINQ)

Task:

Fetch all students and display in view.

Concept:

— LINQ queries

Answer:

To fetch data from database in EF Core, we use LINQ queries. LINQ helps us read, filter, sort data easily from tables.

CONTROLLER

```
using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Data;
using StudentMvcApp.Models;
using System.Linq;
namespace StudentMvcApp.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var students = _context.Students
                .OrderBy(s => s.Name) // LINQ sorting
                .ToList();

            return View(students);
        }
    }
}
```



```
}  
}
```

Index.cshtml

```
@model IEnumerable<StudentMvcApp.Models.Student>  
<h2>Student List</h2>  
<table border="1">  
  <tr>  
    <th>Name</th>  
    <th>Age</th>  
  </tr>  
  @foreach (var s in Model)  
  {  
    <tr>  
      <td>@s.Name</td>  
      <td>@s.Age</td>  
    </tr>  
  }  
</table>
```

◆ 20. Update s Delete (CRUD)

Task:

- Edit student details
- Delete a student

Concept:

— Full CRUD lifecycle

Answer:

CRUD = Create, Read, Update, Delete

- **Update** → Edit existing student
- **Delete** → Remove student from database

EF Core methods used:

- Update → `_context.Update()`
- Delete → `_context.Remove()`

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentMvcApp.Data;
using StudentMvcApp.Models;
namespace StudentMvcApp.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;
        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }
        public IActionResult Index()
        {
            var students = _context.Students.ToList();
            return View(students);
        }

        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Create(Student s)
        {
            _context.Students.Add(s);
            _context.SaveChanges();
            return RedirectToAction("Index");
        }
        public IActionResult Edit(int id)
        {
            var student = _context.Students.Find(id);
            return View(student);
        }
        [HttpPost]
        public IActionResult Edit(Student s)
        {
            _context.Students.Update(s);
            _context.SaveChanges();
            return RedirectToAction("Index");
        }
    }
}
```

```

    }
    public IActionResult Delete(int id)
    {
        var student = _context.Students.Find(id);

        if (student != null)
        {
            _context.Students.Remove(student);
            _context.SaveChanges();
        }

        return RedirectToAction("Index");
    }
}
}

```

Index.cshtml

```

@model List<StudentMvcApp.Models.Student>
<h2>Student List</h2>
<a href="/Student/Create">Add Student</a>
<table border="1">
    <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Actions</th>
    </tr>
    @foreach (var s in Model)
    {
        <tr>
            <td>@s.Name</td>
            <td>@s.Age</td>
            <td>
                <a href="/Student/Edit/@s.Id">Edit</a> |
                <a href="/Student/Delete/@s.Id">Delete</a>
            </td>
        </tr>
    }
</table>

```

Edit.cshtml

```
@model StudentMvcApp.Models.Student
<h2>Edit Student</h2>
<form method="post">
  <input type="hidden" name="Id" value="@Model.Id" />
  <label>Name:</label>
  <input type="text" name="Name" value="@Model.Name" />
  <br />
  <label>Age:</label>
  <input type="number" name="Age" value="@Model.Age" />
  <br />
  <button type="submit">Update</button>
</form>
```

🔗 Problem: Send Notification to User

— You are building an app that sends a message to a user.

Now think:

- Today → Email
 - Tomorrow → SMS
 - Later → WhatsApp
-

+ Without Dependency Injection (Tight Coupling)

◆ Code

```
public class NotificationService
{
  public string Send()
  {
    return "Email sent to user";
  }
}
```

```
public class HomeController : Controller
{
  public IActionResult Index()
```

```
{  
    // Direct object creation (BAD)  
    NotificationService service = new NotificationService();  
  
    string result = service.Send();  
  
    return Content(result);  
}  
}
```

+ What's the Problem?

1. Tight Coupling

- Controller is directly dependent on Email service
-

2. No Flexibility

- If you want SMS instead:

NotificationService service = new SmsService(); // need code change

3. Hard to Test

- You can't easily replace service with fake/mock
-

4. Code Maintenance Issue

- Every change → modify controller +
-

With Dependency Injection (Loose Coupling)

Step 1: Create Interface

```
public interface INotificationService  
{
```

```
    string Send();  
}
```

— This is a contract
(Any service must follow this rule)

◆ Step 2: Create Multiple Services

```
public class EmailService : INotificationService  
{  
    public string Send()  
    {  
        return "Email sent to user";  
    }  
}
```

```
public class SmsService : INotificationService  
{  
    public string Send()  
    {  
        return "SMS sent to user";  
    }  
}
```

◆ Step 3: Register in Program.cs

```
builder.Services.AddScoped<INotificationService, EmailService>();
```

— Today → Email
— Tomorrow → just change to:

```
builder.Services.AddScoped<INotificationService, SmsService>();
```

◆ Step 4: Use in Controller

```
public class HomeController : Controller  
{  
    private readonly INotificationService _service;
```

```
// Injected automatically
public HomeController(INotificationService service)
{
    _service = service;
}

public IActionResult Index()
{
    string result = _service.Send();
    return Content(result);
}
}
```

● What Just Happened?

— Controller DOES NOT know:

- EmailService +
- SmsService +

— It only knows:

"I need something that can send notification"

— ASP.NET decides:

"I will give EmailService"

Key Difference (Important)

Without DI +

With DI



Controller creates service Framework provides service

Tight coupling

Loose coupling

Hard to change

Easy to switch

Not testable

Testable

Without DI ➕

With DI ■

Poor design

Clean architecture

📁 Simple Flow

Without DI:

Controller → new EmailService()

With DI:

Controller → asks for INotificationService

↓

ASP.NET → gives EmailService

— Change this ONE LINE:

```
builder.Services.AddScoped<INotificationService, SmsService>();
```

🔄 Entire app switches from Email → SMS

— NO controller change needed

📌 Final Understanding (Exam Answer Style)

Dependency Injection means:

Instead of creating objects inside a class, we provide them from outside to reduce dependency and improve flexibility.

Answer:

1. INTERFACE

Services/INotificationService.cs

```
namespace StudentMvcApp.Services
{
    public interface INotificationService
    {
        string Send();
    }
}
```



```
}  
}
```

2. EMAIL SERVICE

Services/EmailService.cs

```
namespace StudentMvcApp.Services  
{  
    public class EmailService : INotificationService  
    {  
        public string Send()  
        {  
            return "Email sent to user";  
        }  
    }  
}
```

3. SMS SERVICE

Services/SmsService.cs

```
namespace StudentMvcApp.Services  
{  
    public class SmsService : INotificationService  
    {  
        public string Send()  
        {  
            return "SMS sent to user";  
        }  
    }  
}
```

4. CONTROLLER

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;  
using StudentMvcApp.Services;  
namespace StudentMvcApp.Controllers  
{  
    public class HomeController : Controller  
    {  
        private readonly INotificationService _service;
```

```

    public HomeController(INotificationService service)
    {
        _service = service;
    }

    public IActionResult Index()
    {
        string result = _service.Send();
        return Content(result);
    }
}

```

5. PROGRAM.CS (VERY IMPORTANT)

Program.cs

```

using StudentMvcApp.Services;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<INotificationService, EmailService>();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();

```